BIL 322 SİSTEM PROGRAMLAMA

(BIL 322 System Programming)

Ara Sınav — Hafta 1–7 (Thread'ler Dahil)

(Midterm Exam — Weeks 1–7, Including Threads)

ÖĞRETİM ÜYESİ NÜSHASI: ÇÖZÜMLER, PUANLAMA VE BÖLÜM NOTLARI

(Instructor version: solutions, rubric, and section grades)

Süre: 60 dakika **Toplam:** 100 puan **Soru sayısı:** 20

Açıklamalar *(Instructions)*

- Bütün soruları cevaplayınız. Aksi belirtilmedikçe çoktan seçmeli soruların tek bir en doğru cevabı vardır. *(Answer all questions. Multiple-choice questions have one best answer unless stated otherwise.)*
- Tek bir A4 boyutunda kopya kâğıdı (cheat sheet) kullanılabilir. *(One A4 cheat-sheet paper is permitted.)*
- Kod çıktısı ve kısa cevaplı sorularda, kısmi puan verilebilmesi için çözüm adımlarınızı yeterince gösteriniz. *(For code-output and short-answer questions, show enough reasoning to allow partial credit.)*
- Soruda aksi belirtilmedikçe little-endian x86-64 bir Linux sistemi, C11/POSIX ve bütün sistem/kütüphane çağrılarının başarılı olduğunu varsayınız. *(Unless a question states otherwise, assume Linux on a little-endian x86-64 system, C11/POSIX, and successful system/library calls.)*
- Tam bir program yazmanız istenmemektedir. Bazı sorularda yalnızca bir veya iki deyim yazmanız istenir. *(No complete program writing is required. Some items ask for one or two statements only.)*
- Sorular Türkçedir; parantez içindeki İngilizce metin yalnızca yardım amaçlıdır. *(Questions are in Turkish; the English text in parentheses is provided for reference.)*

Bölüm Puan Özeti *(Section Grade Summary)*

Her öğrenci için alınan puanlar son sütuna yazılır. Bölüm yüzdesi = (bölümde alınan puan / bölümün toplam puanı) × 100.

Bölüm	Ölçülen öğrenme çıktısı *(Learning outcome assessed)*	Sorular	Puan	Alınan
A	**LO1 (Application):** Ability to use C development tools such as gcc, make, Git, GDB, Valgrind, and ThreadSanitizer.	1–2	8	
B	**LO2 (Comprehension):** Ability to explain object files, static and dynamic linking, symbols, and program loading.	3–4	8	
C	**LO3 (Comprehension):** Ability to explain C memory layout, pointers, structs, alignment, and binary file I/O.	5–9	20	
D	**LO4 (Application):** Ability to use Unix system calls for file I/O, directory access, and process control.	10–12	16	
E	**LO5 (Application):** Ability to design and implement simple shells or pipeline runners using fork, exec, wait, pipes, and redirection.	13–14	14	
F	**LO6 (Application):** Ability to implement a simple memory allocator.	15–17	16	
G	**LO7 (Application):** Ability to write concurrent, multithreaded C programs using POSIX threads.	18–19	12	
H	**LO9 (Analysis; partial):** Ability to analyze race conditions and diagnose them with tools such as ThreadSanitizer and GDB. Deadlock and livelock are outside this midterm.	20	6	
	Toplam *(Total)*		**100**	

Kapsam notu (scope note): Section H assesses only race conditions and their diagnosis from LO9. LO8, deadlock, livelock, mutexes, condition variables, semaphores, barriers, and thread pools are outside this midterm.

Bölüm A: Development Tools, Make, and Debugging — 8 puan

Öğrenme Çıktısı 1 | *Kapsam: Hafta 1 ve 3*

[4 puan] 1. Bir proje, aşağıdaki Makefile kurallarıyla daha önce başarıyla derlenmiştir:

(A project has already been built successfully with the following Makefile rules:)

```
bench: bench.o vector_ops.o
        $(CC) bench.o vector_ops.o -o bench

bench.o: bench.c vector_ops.h
        $(CC) -c bench.c -o bench.o

vector_ops.o: vector_ops.c vector_ops.h
        $(CC) -c vector_ops.c -o vector_ops.o
```

Daha sonra yalnızca `vector_ops.h` düzenleniyor. Ardından hedef belirtmeden `make` çalıştırıldığında ne olur?

(Only `vector_ops.h` is then edited. What happens when `make` is run next, with no target given?)

A. Hiçbir `.c` dosyası değişmediği için hiçbir şey yeniden derlenmez. *(Nothing is rebuilt because no `.c` file changed.)*

B. Yalnızca `vector_ops.o` yeniden derlenir. *(Only `vector_ops.o` is rebuilt.)*

C. Her iki nesne dosyası da yeniden derlenir, ardından `bench` yeniden bağlanır (relink). *(Both object files are rebuilt, and then `bench` is relinked.)*

D. Yalnızca son bağlama (link) adımı çalışır. *(Only the final link step runs.)*

Solution: Her iki nesne dosyası da `vector_ops.h` dosyasına bağımlı olduğu için ikisi de güncelliğini yitirir. Bunların yeniden derlenmesi `bench` dosyasını da güncel olmaktan çıkarır; dolayısıyla `bench` yeniden bağlanır.

(Both object files depend on `vector_ops.h`, so both become out of date. Rebuilding them makes `bench` out of date as well, so it is relinked.)

Rubric: 4 points for choice C; 0 otherwise.

[4 puan] 2. Valgrind, bir C programı çalışırken aşağıdaki raporu veriyor:

(Valgrind reports the following while running a C program:)

```
==142== Invalid read of size 4
==142==    at 0x1091A2: sum (vector.c:12)
```

En doğru yorum hangisidir?

(What is the best interpretation?)

A. Programda kesinlikle bellek sızıntısı (memory leak) vardır, ancak geçersiz erişim yoktur. *(The program definitely has a memory leak, but no invalid access.)*

B. Bağlayıcı (linker), `sum` sembolünü bulamamaktadır. *(The linker cannot find `sum`.)*

C. Program, erişmemesi gereken bir adresten 4 byte okumaktadır. *(The program reads four bytes from an address it must not access.)*

D. İki thread arasında kesinlikle bir data race vardır. *(Two threads necessarily have a data race.)*

Solution: "Invalid read of size 4", geçersiz bir adresten 4 byte okunduğu anlamına gelir; örneğin dizi sınırı dışına erişim (out-of-bounds) veya serbest bırakılmış belleğin kullanılması (use-after-free) olabilir. Bu mesaj tek başına bir bellek sızıntısı raporu değildir.

("Invalid read of size 4" means a four-byte read from an invalid address, for example an out-of-bounds access or a use-after-free. It is not, by itself, a leak report.)

Rubric: 4 points for choice C; 0 otherwise.

Bölüm B: Object Files, Linking, Symbols, and Loading — 8 puan

Öğrenme Çıktısı 2 | *Kapsam: Hafta 1*

[4 puan] 3. `nm -A bench.o vector_ops.o` komutu aşağıdaki satırları üretiyor:

(The command `nm -A bench.o vector_ops.o` produces these lines:)

```
bench.o:                 U dot_product
vector_ops.o:0000000000000000 T dot_product
```

Bu satırlar ne anlama gelir?

(What do these lines mean?)

A. Her iki nesne dosyası da `dot_product` için çalıştırılabilir bir tanım içerir. *(Both object files contain an executable definition of `dot_product`.)*

B. `bench.o` bozulmuştur ve bağlanamaz. *(`bench.o` is corrupted and cannot be linked.)*

C. `bench.o`, `dot_product` sembolüne referans verir; `vector_ops.o` ise onu text bölümünde (`.text`) tanımlar. ***(`bench.o` references `dot_product`, while `vector_ops.o` defines it in the text section.)***

D. `dot_product`, her iki dosyada da ilk değer atanmamış bir global değişkendir. *(`dot_product` is an uninitialized global variable in both files.)*

Solution: `U`, başka bir nesne dosyasından veya kütüphaneden çözülmesi gereken tanımsız (undefined) bir sembolü gösterir. `T`, text (kod) bölümündeki bir tanımı gösterir. Bağlama sırasında `bench.o` içindeki referans, `vector_ops.o` içindeki tanım ile çözülür.

(`U` marks an undefined symbol that must be resolved from another object or library. `T` marks a definition in the text/code section. Linking resolves the reference in `bench.o` using the definition in `vector_ops.o`.)

Rubric: 4 points for choice C; 0 otherwise.

[4 puan] 4. Statik bağlama, dinamik bağlama ve program yükleme arasındaki ilişkiyi en doğru açıklayan ifade hangisidir?

(Which statement best describes the relationship among static linking, dynamic linking, and program loading?)

A. Statik bağlama, kütüphane kodunu çalıştırılabilir dosyaya kopyalamaz; yükleyici (loader) bütün fonksiyonları çalışma zamanında kaynak koddan derler. *(Static linking copies no library code into the executable; the loader compiles all functions from source code at run time.)*

B. Dinamik bağlama, bütün kütüphane kodunu bağlama zamanında çalıştırılabilir dosyaya kopyalar; statik bağlama ise yalnızca paylaşımlı nesne (`.so`) üretir. *(Dynamic linking*

copies all library code into the executable at link time; static linking only produces a shared object (`.so`).)

C. Statik bağlama, gereken kütüphane kodunu çalıştırılabilir dosyanın içine yerleştirir; dinamik bağlamada ise çalıştırılabilir dosya yalnızca paylaşımlı kütüphane bağımlılıklarını kaydeder ve dinamik yükleyici bu kütüphaneleri program başlatılırken yükler. ***(Static linking places the required library code inside the executable; with dynamic linking the executable only records its shared-library dependencies, and the dynamic loader loads those libraries when the program is started.)***

D. Statik ve dinamik bağlama yalnızca derleyicinin optimizasyon seviyesini değiştirir; sembol çözümlemeye etkileri yoktur. *(Static and dynamic linking only change the compiler optimization level; they do not affect symbol resolution.)*

Solution: Statik bağlama, gereken kütüphane kodunu bağlama zamanında çalıştırılabilir dosyanın içine alır. Dinamik bağlamada çalıştırılabilir dosya, paylaşımlı kütüphane bağımlılıklarına referans taşır; dinamik yükleyici bu kütüphaneleri yükler ve kalan sembol çözümleme/relocation işlemlerini tamamlar.

(Static linking incorporates the required library code into the executable at link time. With dynamic linking, the executable refers to shared-library dependencies; the dynamic loader loads those libraries and completes the remaining symbol resolution/relocation work.)

Rubric: 4 points for choice C; 0 otherwise.

Bölüm C: C Memory, Pointers, Structs, and Binary Data — 20 puan

Öğrenme Çıktısı 3 | *Kapsam: Hafta 2–3*

[4 puan] 5. Bu kod parçası tam olarak ne yazdırır?

(What exactly does this code fragment print?)

```c
int a[] = {10, 20, 30};
int *p = a;
p++;
printf("%d %d\n", *p, p[1]);
(*p)++;
printf("%d\n", a[1]);
```

Solution: Çıktı:

(The output is:)

```
20 30
21
```

`p++` işleminden sonra `p`, `a[1]` elemanını gösterir. Bu nedenle `*p` 20, `p[1]` ise `a[2]`, yani 30'dur. `(*p)++` ifadesi `a[1]` değerini 21'e çıkarır.

(After `p++`, `p` points to `a[1]`. Thus `*p` is 20 and `p[1]` is `a[2]`, or 30. The expression `(*p)++` increments `a[1]` to 21.)

Rubric: 2 points for the first line (1 per value) and 2 points for the second line.

[4 puan] 6. Her öğe için en uygun bellek bölgesini yazınız: **.data**, **.bss**, **stack** veya **heap**.

*(For each item, write the most appropriate memory region: **.data**, **.bss**, **stack**, or **heap**.)*

```c
int initialized = 5;
int uninitialized;

int main(void) {
    int local;
    int *p = malloc(sizeof *p);
    /* ... */
}
```

(a) `initialized` _____

(b) `uninitialized` _____

(c) `local` _____

(d) `*p` ile erişilen bellek alanı (`p` değişkeninin kendisi değil) *(The storage accessed through `*p` (not the variable `p` itself))* _____

Solution: (a) `.data`; (b) `.bss`; (c) stack; (d) heap. `p` pointer değişkeninin kendisi stack üzerindedir, ancak gösterdiği, `malloc` ile ayrılmış alan heap üzerindedir.

((a) `.data`; (b) `.bss`; (c) stack; (d) heap. The pointer variable `p` itself is on the stack, but the allocated storage it points to is on the heap.)

Rubric: 1 point for each correct region.

[4 puan] 7. `char` için boyut/hizalama değerinin 1, `uint32_t` için 4 ve `double` için 8 olduğunu varsayınız. Aşağıdaki yapıyı inceleyiniz:

(Assume `char` has size/alignment 1, `uint32_t` has size/alignment 4, and `double` has size/alignment 8. Consider:)

```c
typedef struct {
    char tag;
    uint32_t count;
    double value;
} Record;
```

(a) `count` alanının offset değerini, (b) `value` alanının offset değerini, (c) `sizeof(Record)` değerini ve (d) bir `Record` içindeki toplam padding byte sayısını bulunuz.

(Give (a) the offset of `count`, (b) the offset of `value`, (c) `sizeof(Record)`, and (d) the total number of padding bytes in one `Record`.)

Solution: (a) 4; (b) 8; (c) 16 byte; (d) 3 padding byte. `count` alanını offset 4'e hizalamak için `tag` alanından sonra 3 byte bırakılır. `value` alanı offset 8'de zaten hizalıdır ve toplam boyut olan 16, 8'in katı olduğu için sonda (tail) padding gerekmez.

((a) 4; (b) 8; (c) 16 bytes; (d) 3 padding bytes. There are three bytes after `tag` to align `count` to offset 4. `value` is already aligned at offset 8, and the total size 16 is a multiple of 8, so no tail padding is needed.)

Rubric: 1 point for each correct value.

[4 puan] 8. Little-endian bir makinede:

(On a little-endian machine:)

```
uint32_t x = 0x12345678;
unsigned char b[4];
memcpy(b, &x, 4);
```

(a) `b[0]`, `b[1]`, `b[2]` ve `b[3]` değerlerini onaltılık (hexadecimal) olarak yazınız. (b) Taşınabilir (portable) bir ikili dosya formatında bir alan tanımlanırken `uint32_t` neden `unsigned long` yerine tercih edilir?

((a) Write `b[0]`, `b[1]`, `b[2]`, and `b[3]` in hexadecimal. (b) Why is `uint32_t` preferable to `unsigned long` when defining a field of a portable binary file format?)

Solution: (a) `b[0] = 0x78`, `b[1] = 0x56`, `b[2] = 0x34`, `b[3] = 0x12`. Little-endian makinede en düşük anlamlı byte (least-significant byte) önce saklanır.

(b) `uint32_t`, tanımlı olduğu her platformda tam olarak 32 bit genişliğindedir; `unsigned long` genişliği ise platforma/ABI'ye göre değişir (ör. x86-64 Linux'ta 64 bit, Windows'ta 32 bit). Sabit genişlikli alanlar ikili düzeni (binary layout) açık ve sabit hâle getirir.

((a) `b[0] = 0x78`, `b[1] = 0x56`, `b[2] = 0x34`, `b[3] = 0x12`. The least-significant byte is stored first on a little-endian machine. (b) `uint32_t` has an exact 32-bit width wherever it is provided, whereas the width of `unsigned long` varies across platforms/ABIs (e.g., 64 bits on x86-64 Linux, 32 bits on Windows). Fixed-width fields make a binary layout explicit.)

Rubric: 3 points for all four bytes in the correct order; 1 point (instead of 3) if the four bytes are given in big-endian order (`12 34 56 78`); 1 point for exact/fixed width across platforms.

[4 puan] 9. Aşağıdaki kodu inceleyiniz:

(Consider:)

```
#define SQR(x) ((x) * (x))
int i = 3;
int y = SQR(i++);
```

`y` değerinin kesinlikle 9 olacağını söylemek neden doğru değildir? Bu işlem için daha güvenli bir C yapısı (construct) söyleyiniz.

(Why is it wrong to claim that `y` must be 9? Name a safer C construct for this operation.)

Solution: Makro, `((i++) * (i++))` biçiminde açılır (expand). Argüman iki kez değerlendirildiği için `i` aynı ifade içinde, aralarında sıralama olmadan (unsequenced) iki kez değiştirilir; bu tanımsız davranıştır (undefined behavior). Makro argümanının etrafındaki parantezler argümanın iki kez değerlendirilmesini önlemez.

Daha güvenli yapı, argümanını yalnızca bir kez değerlendiren normal bir fonksiyon veya `static inline` fonksiyondur; örneğin `static inline int sqr_int(int x) { return x * x; }`.

(The macro expands to `((i++) * (i++))`. The argument is evaluated twice, producing two unsequenced modifications of `i`; the behavior is undefined. Parentheses around the macro argument do not prevent duplicate evaluation. A safer construct is a normal or `static inline` function, such as `static inline int sqr_int(int x)`, which evaluates its argument once.)

Rubric: 2 points for identifying double evaluation *and* undefined behavior (1 point if the answer only shows double evaluation, e.g., "it becomes $3 \times 4 = 12$", without mentioning

undefined behavior); 2 points for a normal or inline function (or an equivalent single-evaluation approach).

Bölüm D: File I/O and Process Control — 16 puan

Öğrenme Çıktısı 4 | *Kapsam: Hafta 3–4*

[6 puan] 10. `fd`, `O_RDONLY` ile açılmış bir dosyayı gösteriyor. Dosyanın içeriği tam olarak 6 byte'lık `abcdef` dizisidir (sonunda newline yoktur) ve dosya offset'i başlangıçta 0'dır. Tam olarak ne yazdırılır?

(`fd` refers to a file opened with `O_RDONLY`. The file contains exactly the 6 bytes `abcdef` (no trailing newline), and its file offset is initially 0. What exactly is printed?)

```
char buf[4] = {0, 0, 0, 0};
ssize_t n = read(fd, buf, 3);
printf("%zd:%s\n", n, buf);

lseek(fd, -2, SEEK_END);
n = read(fd, buf, 2);
buf[n] = '\0';
printf("%zd:%s\n", n, buf);
```

Solution:

```
3:abc
2:ef
```

İlk `read` 3 döndürür; `buf[3]` başlangıçta 0 olduğu için `buf` null ile sonlanan bir string olarak kalır. `lseek`, offset'i dosya sonundan (EOF) 2 byte önceye, yani `e` karakterine konumlandırır; ikinci `read` 2 döndürür.

(The first read returns 3 and `buf` stays null-terminated because `buf[3]` was initially zero. The `lseek` positions the offset two bytes before EOF, at `e`; the second read returns 2.)

Rubric: 3 points per line: 1 for the return value and 2 for the correct bytes/string.

[4 puan] 11. `data.txt` dosyasının içeriği tam olarak `AAAABBBBCCCC` dizisidir. Parent süreç, `fork` çağırmadan önce dosyayı açıyor:

(`data.txt` contains exactly `AAAABBBBCCCC`. The parent opens the file before calling `fork`:)

```
int fd = open("data.txt", O_RDONLY);
char buf[5] = {0};

pid_t pid = fork();
if (pid == 0) {
    read(fd, buf, 4);
    _exit(0);
}
waitpid(pid, NULL, 0);
read(fd, buf, 4);
printf("%s\n", buf);
```

Parent ne yazdırır?

(What does the parent print?)

A. `AAAA`; çünkü `fork` sonrasında her süreç kendi dosya offset'ine sahiptir. *(`AAAA`, because after `fork` each process has its own file offset.)*

B. `BBBB`; çünkü iki süreçteki `fd` aynı açık dosya tanımını (open file description) ve dolayısıyla aynı offset'i paylaşır. ***(`BBBB`, because `fd` in both processes refers to the same open file description and therefore shares one offset.)***

C. Hiçbir şey okunmaz; çünkü child sonlandığında `fd` parent'ta da kapanır. *(Nothing is read, because `fd` is also closed in the parent when the child exits.)*

D. Çıktı belirsizdir; çünkü `fork` sonrasında hangi sürecin önce çalışacağı bilinemez. *(The output is unpredictable, because it is unknown which process runs first after `fork`.)*

Solution: `fork` sonrasında child, parent'ın file descriptor tablosunun bir kopyasını alır, ancak iki tablo da aynı açık dosya tanımını (open file description) gösterir. Dosya offset'i bu ortak yapıda tutulur. Parent, `waitpid` ile child'ın okumasının bitmesini beklediği için child offset'i 4'e ilerletmiş olur ve parent `BBBB` okur.

(After `fork`, the child gets a copy of the parent's file-descriptor table, but both tables point to the same open file description, which holds the file offset. Because the parent waits for the child with `waitpid`, the child has already advanced the shared offset to 4, so the parent reads `BBBB`.)

Rubric: 4 points for choice B; 0 otherwise.

[6 puan] 12. `fork` ve `waitpid` çağrılarının başarılı olduğunu varsayınız. (a) Tam olarak ne yazdırılır? (b) `status` değeri, child'ın bir sinyal ile sonlandığını mı gösterir? `WEXITSTATUS(status)` değerinin anlamlı olduğundan emin olmak için önce hangi makro kontrol edilmelidir?

(Assume `fork` and `waitpid` succeed. (a) What exactly is printed? (b) Does `status` indicate that the child was terminated by a signal? Which macro should be checked first to make sure that `WEXITSTATUS(status)` is meaningful?)

```c
int status;
printf("A");
fflush(stdout);

pid_t pid = fork();
if (pid == 0) {
    printf("C");
    fflush(stdout);
    _exit(3);
}

waitpid(pid, &status, 0);
printf("P%d\n", WEXITSTATUS(status));
```

Solution: (a) Çıktı `ACP3` ve ardından bir newline'dır. `fork`'tan önce `A` boşaltıldığı (flush) için child onu tekrar yazdırmaz; parent child'ı beklediği için sıralama deterministiktir.

(b) Hayır. Child, `_exit(3)` ile normal biçimde sonlanır; dolayısıyla `WIFEXITED(status)` doğrudur (true). Önce bu makro kontrol edilmelidir; ancak o zaman `WEXITSTATUS(status)` çıkış kodunu (3) verir. Sıfırdan farklı bir çıkış kodu, sinyal ile sonlanma anlamına gelmez.

((a) The output is `ACP3` followed by a newline. `A` was flushed before `fork`, so the child does not print it again, and the parent waits for the child, so the ordering is deterministic. (b) No. The child terminated normally through `_exit(3)`, so `WIFEXITED(status)` is true. `WIFEXITED(status)` should be checked

first; only then does `WEXITSTATUS(status)` give the exit code (3). A nonzero exit code does not imply signal termination.)

Rubric: 4 points for the exact output and ordering; 1 for normal exit/not a signal; 1 for `WIFEXITED`.

Bölüm E: Shell Mechanisms, Pipes, and Redirection — 14 puan

Öğrenme Çıktısı 5 | *Kapsam: Hafta 4–5*

[6 puan] 13. Aşağıdaki çağrıların hepsi başarılıdır.

(All of the following calls succeed.)

```
int logfd = open("task.log", O_WRONLY | O_CREAT | O_TRUNC, 0600);
dup2(logfd, STDOUT_FILENO);
close(logfd);
printf("finished\n");
```

(a) `finished` metni nereye yazılır? (b) `dup2` çağrısından sonra file descriptor 1 neyi gösterir? (c) `close(logfd)` çağrısı neden çıktıyı engellemez?

((a) Where is the text `finished` written? (b) After the `dup2` call, what does file descriptor 1 refer to? (c) Why does `close(logfd)` not prevent the output?)

Solution: (a) Çıktı `task.log` dosyasına yazılır. (b) `dup2`, `STDOUT_FILENO` (descriptor 1) değerinin `logfd` ile aynı açık dosya tanımını (open file description), yani `task.log` dosyasını göstermesini sağlar. (c) `logfd` kapatıldığında yalnızca o descriptor numarası kaldırılır; descriptor 1 aynı açık dosyayı göstermeye devam eder ve kullanılabilir kalır.

((a) The output is written to `task.log`. (b) `dup2` makes `STDOUT_FILENO` (descriptor 1) refer to the same open file description as `logfd`, i.e., `task.log`. (c) Closing `logfd` removes only that descriptor number; descriptor 1 still refers to the open file and remains usable.)

Rubric: 2 points for (a) `task.log`; 2 for (b) descriptor 1 referring to the log file/same open file description; 2 for (c) independent descriptor numbers/stdout remaining open.

[8 puan] 14. Bir parent süreç, `grep` komutunun çıktısının `awk` komutuna girdi olduğu iki aşamalı bir pipeline kuruyor. Parent, `pipe(pipefd)` çağrısından sonra iki child oluşturmuştur; her iki child da `dup2` ile yönlendirmeyi doğru yapıyor ve kullanmadığı pipe descriptor'larını kapatıyor. Parent ise ardından şunları yapıyor:

(A parent builds a two-stage pipeline in which the output of `grep` is the input of `awk`. After `pipe(pipefd)`, the parent has created both children; both children redirect correctly with `dup2` and close their unused pipe descriptors. The parent then does:)

```
close(pipefd[0]);
wait(NULL);
wait(NULL);
```

Pipeline askıda kalıyor (hang). (a) Bunun nedenini açıklayınız. (b) Parent'ta sorunu düzelten tek deyimi (statement) ve nereye konması gerektiğini yazınız. (c) Her süreç, kullanmadığı pipe uçlarını neden kapatmalıdır?

(The pipeline hangs. (a) Explain why. (b) Write the one statement that fixes it in the parent, and where it goes. (c) Why must every process close the pipe ends it does not use?)

Solution: (a) `awk`, EOF görene kadar okur. Bir pipe'ın okuma ucunda EOF, ancak bütün yazma ucu descriptor'ları kapatıldığında oluşur. Parent hâlâ `pipefd[1]` descriptor'ını açık tuttuğu için `awk` EOF göremez ve sonlanamaz; parent, kendisini bekleyen bir child'ı beklemiş olur.

(b) Parent, `wait` çağrılarından önce şu deyimi çalıştırmalıdır:

```
close(pipefd[1]);
```

(c) Kullanılmayan okuma uçları kapatılmalıdır ki okuyucu kalmadığında yazma işlemleri broken pipe (`SIGPIPE`/`EPIPE`) bildirebilsin; kullanılmayan yazma uçları kapatılmalıdır ki okuyucular EOF alabilsin. Herhangi bir süreçte açık kalan bir descriptor, ilgili pipe ucunu canlı tutar.

((a) `awk` reads until EOF. EOF on a pipe's read end occurs only when all write-end descriptors are closed. The parent still holds `pipefd[1]` open, so `awk` cannot see EOF and cannot exit; the parent waits for a child that is effectively waiting for the parent. (b) Before the waits, the parent must execute `close(pipefd[1]);`. (c) Unused read ends must be closed so writes can report a broken pipe (`SIGPIPE`/`EPIPE`) when no reader exists; unused write ends must be closed so readers can receive EOF. An open descriptor in any process keeps the corresponding pipe end alive.)

Rubric: 4 points for the parent's open write end preventing EOF (and the resulting circular wait); 2 for the exact close statement placed before the waits; 2 for descriptor lifetime/EOF (broken-pipe behavior optional).

Bölüm F: Memory Allocation and Allocator Design — 16 puan

Öğrenme Çıktısı 6 | *Kapsam: Hafta 6*

[4 puan] 15. Bu deyim neden bellek sızıntısına yol açabilir ve hangi programlama kalıbı bu sorunu önler?

(Why can this statement cause a memory leak, and what programming pattern prevents the problem?)

```
p = realloc(p, new_size);
```

Solution: `realloc` başarısız olursa `NULL` döndürür, ancak orijinal blok ayrılmış (allocated) olarak kalır. Sonucu doğrudan `p`'ye atamak, eski bloğu gösteren tek pointer'ın kaybolmasına neden olur. Geçici bir pointer kullanılmalı, `NULL` kontrolü yapılmalı ve yalnızca başarı durumunda `p`'ye atama yapılmalıdır.

(If `realloc` fails, it returns `NULL` but the original block remains allocated. Assigning directly to `p` loses the only pointer to that old block. Use a temporary pointer, check it for `NULL`, and assign back to `p` only on success.)

Rubric: 2 points for "failure leaves the old block allocated and the pointer is lost"; 2 for the temporary-pointer/check-before-assignment pattern.

[6 puan] 16. Bir bellek tahsisçisinin (allocator) adrese göre sıralı serbest listesinde (free list) sırasıyla 8, 20 ve 12 KiB boyutlarında boş bloklar (hole) bulunuyor. (a) 10 KiB'lık bir istek için **first fit** hangi bloğu seçer? (b) Aynı istek için **best fit** hangi bloğu seçer? (c) Farklı bir heap durumunda, bitişik

olmayan iki 8 KiB'lık boş blok, toplamda 16 KiB boş alan olmasına rağmen 12 KiB'lık bir isteği karşılayamıyor. Bu durumun adı nedir?

*(An allocator's free list is ordered by address and contains holes of sizes 8, 20, and 12 KiB, in that order. (a) Which hole does **first fit** choose for a 10 KiB request? (b) Which hole does **best fit** choose for the same request? (c) In a different heap state, two nonadjacent 8 KiB holes cannot satisfy a 12 KiB request even though 16 KiB is free in total. What is this called?)*

Solution: (a) 20 KiB'lık blok; çünkü yeterince büyük olan ilk blok budur. (b) 12 KiB'lık blok; çünkü yeterli olan en küçük blok budur. (c) Dış parçalanma (external fragmentation).

((a) The 20 KiB hole, because it is the first hole large enough. (b) The 12 KiB hole, because it is the smallest sufficient hole. (c) External fragmentation.)

Rubric: 2 points for each correct answer.

[6 puan] 17. Üç bloklu, sabit blok boyutlu bir havuz (pool), Lab 6'daki gibi serbest listeyi boş blokların içinde tutuyor: boş bir bloğun ilk byte'ları, bir sonraki boş bloğun adresini saklar.

(A fixed-size pool with three blocks keeps its free list inside the free blocks, as in Lab 6: the first bytes of a free block store the address of the next free block.)

```c
typedef struct FreeBlock { struct FreeBlock *next; } FreeBlock;
FreeBlock *free_head;   /* fresh pool: free_head -> b0 -> b1 -> b2 -> NULL */

void *pool_alloc(void) {
    FreeBlock *b = free_head;
    if (b == NULL) return NULL;
    free_head = b->next;            /* pop */
    return b;
}

void pool_free(void *ptr) {
    FreeBlock *b = ptr;
    b->next = free_head;            /* push */
    free_head = b;
}
```

Yeni oluşturulmuş havuz üzerinde program şunları yapıyor:

(Starting from the fresh pool, the program performs:)

```c
p0 = pool_alloc();  p1 = pool_alloc();
pool_free(p0);      p2 = pool_alloc();
```

Blokları `b0`, `b1`, `b2` veya `NULL` olarak yazınız. (a) `p0` ve `p1`'in gösterdiği blokları, (b) bu iki ayırmadan sonra `free_head` değerini, (c) `pool_free(p0)` çağrısından hemen sonra `free_head` değerini ve `b0` içinde saklanan `next` değerini, (d) `p2`'nin gösterdiği bloğu veriniz.

(Answer with `b0`, `b1`, `b2`, or `NULL`. Give (a) the blocks that `p0` and `p1` point to, (b) `free_head` after those two allocations, (c) `free_head` and the `next` value stored in `b0` immediately after `pool_free(p0)`, and (d) the block that `p2` points to.)

Solution: (a) `p0 = b0`, `p1 = b1`. (b) `free_head = b2` (liste: `b2 -> NULL`). (c) `free_head = b0` ve `b0->next = b2` (liste: `b0 -> b2 -> NULL`). (d) `p2 = b0`; serbest liste LIFO çalıştığı için son bırakılan blok ilk yeniden verilir (son durumda `free_head = b2`).

((a) `p0 = b0`, `p1 = b1`. (b) `free_head = b2` (list: `b2 -> NULL`). (c) `free_head = b0` and `b0->next = b2` (list: `b0 -> b2 -> NULL`). (d) `p2 = b0`; the free list is LIFO, so the most recently freed block is handed out first (finally `free_head = b2`).)

Rubric: 1 point each for `p0`, `p1`, the head after two allocations, the head after the free, `b0->next`, and `p2`.

Bölüm G: POSIX Threads and Thread Lifecycle — 12 puan

Öğrenme Çıktısı 7 | *Kapsam: Hafta 7*

[5 puan] 18. Aynı süreç içindeki thread'ler için her öğeyi **paylaşılan** (bütün thread'ler aynı öğeyi kullanır) veya **thread'e özel** (her thread'in kendi kopyası vardır) olarak sınıflandırınız:

*(For threads in the same process, classify each item as **shared** (all threads use the same item) or **per-thread** (each thread has its own):)*

(a) Bir global değişken *(A global variable)* _____

(b) İki thread'in de pointer'ını tuttuğu bir heap bloğu *(A heap block to which both threads hold a pointer)* _____

(c) Thread fonksiyonu içinde tanımlanmış yerel (local) bir değişken *(A local variable declared inside the thread function)* _____

(d) Program sayacı (program counter) ve register değerleri *(The program counter and register values)*

(e) Sürecin açık file descriptor tablosu *(The process's open file-descriptor table)* _____

Solution: (a) paylaşılan; (b) paylaşılan; (c) thread'e özel; (d) thread'e özel; (e) paylaşılan. Thread'lerin ayrı kontrol akışları ve stack'leri vardır, ancak sürecin adres alanını ve kaynaklarını paylaşırlar.

((a) shared; (b) shared; (c) per-thread; (d) per-thread; (e) shared. Threads have separate control flow and stacks but share the process address space and process resources.)

Rubric: 1 point for each correct classification.

[7 puan] 19. `worker` fonksiyonu, argümanı üzerinden bir tamsayı okuyor (`int id = *(int *)arg;`). `main` thread'i, `tid` dizisinin yanında tanımlı `int i;` ile dört thread oluşturuyor ve hepsini join ediyor.

(The `worker` function reads an integer through its argument (`int id = *(int *)arg;`). The main thread, with `int i;` declared next to the array `tid`, creates four threads and joins all of them.)

```
for (i = 0; i < 4; i++)
    pthread_create(&tid[i], NULL, worker, &i);

for (i = 0; i < 4; i++)
    pthread_join(tid[i], NULL);
```

(a) `&i` geçirmek neden güvenli değildir? (b) `main` içinde tanımlanmış `int ids[4];` dizisini kullanarak ilk döngünün gövdesi yerine geçecek güvenli, iki deyimlik bir kod yazınız. (c) `main` thread'i, worker'ların ürettiği sonuçları kullanmadan önce neden join yapmalıdır?

((a) Why is passing `&i` unsafe? (b) Using an array `int ids[4];` declared in `main`, write a safe two-statement replacement for the body of the first loop. (c) Why must the main thread join before using results produced by the workers?)

Solution: (a) Bütün thread'ler aynı adresi alır. `main` thread'i `i` değerini değiştirmeye devam ederken (ikinci döngüde bile) worker'lar onu okur; bu nedenle worker'lar aynı değeri, daha sonraki bir değeri veya 4'ü görebilir. Senkronizasyonsuz okuma/yazma ayrıca bir data race oluşturur.

(b) Güvenli kod:

```
ids[i] = i;
pthread_create(&tid[i], NULL, worker, &ids[i]);
```

Böylece her thread, farklı ve değeri değişmeyen bir nesne alır.

(c) `pthread_join`, hedef thread'in sonlanmasını bekler. Join, `main` thread'i sonuçları kullanmadan önce worker'ların sonuç üretmeyi bitirmiş olmasını garanti eder.

((a) All threads receive the same address. The main thread keeps changing `i` (even in the join loop) while workers read it, so workers can observe the same value, a later value, or 4; the unsynchronized read/write is also a data race. (b) A safe replacement is `ids[i] = i;` followed by `pthread_create(&tid[i], NULL, worker, &ids[i]);`. Each thread then receives a distinct, stable object. (c) `pthread_join` waits for the target thread to terminate. Joining guarantees that the workers have finished producing their results before the main thread consumes them.)

Rubric: 3 points for the shared loop variable/race explanation; 2 for the two safe statements (1 each); 2 for join as completion/synchronization before result use.

Bölüm H: Race Conditions and Diagnosis — 6 puan

Öğrenme Çıktısı 9 (partial) | *Kapsam: Hafta 7; yalnızca yarış durumu tanılama*

[6 puan] 20. İki thread, `worker` fonksiyonunu birer kez çalıştırıyor; `counter` başlangıçta 0'dır ve `main` thread'i iki thread'i de join ediyor. Mutex veya atomik işlem kullanılmıyor.

(Two threads each run `worker` once; `counter` starts at 0, and the main thread joins both threads. No mutex or atomic operation is used.)

```
int counter = 0;
void *worker(void *arg) {
    counter++;  /* conceptually: load, add 1, store */
    return NULL;
}
```

Thread'ler sırayla (art arda) çalışsaydı sonuç 2 olurdu. (a) Gözlemlenen son değer 1 olabilir mi? Kayıp güncellemeyi (lost update) açıklayan bir load/store sıralaması (interleaving) veriniz ve C düzeyindeki doğruluk sorununun adını belirtiniz. (b) Programı derleyip çalıştırırken bu hatayı otomatik olarak raporlamak için hangi tanılama aracı uygundur?

(If the threads ran one after the other, the result would be 2. (a) Can the observed final value be 1? Give a load/store interleaving that explains a lost update, and name the C-level correctness problem. (b) Which diagnosis tool is appropriate for automatically reporting this bug when compiling and running the program?)

Solution: (a) Evet, 1 değeri kayıp güncellemeye bir örnektir. Olası bir kavramsal sıralama:

1. Thread A, `counter = 0` değerini load eder. *(Thread A loads `counter = 0`.)*

2. Thread B, `counter = 0` değerini load eder. *(Thread B loads `counter = 0`.)*
3. Thread A, 1 değerini store eder. *(Thread A stores 1.)*
4. Thread B, 1 değerini store ederek A'nın güncellemesinin üzerine yazar. *(Thread B stores 1, overwriting A's update.)*

Her iki artırma da 0 değerine dayandığı için yalnızca bir artırma görünür. Daha kesin olarak, senkronize edilmemiş çakışan erişimler bir C data race oluşturur; dolayısıyla programın davranışı tanımsızdır ve belirli bir çıktı garanti edilemez.

(b) **ThreadSanitizer (TSan)** uygundur; program `-fsanitize=thread` ile derlenip bağlandıktan sonra çalıştırıldığında data race raporu verir.

*((a) Yes, 1 is an illustration of a lost update. One possible conceptual interleaving is: Thread A loads 0; Thread B loads 0; Thread A stores 1; Thread B stores 1, overwriting A's update. Both increments were based on 0, so only one increment is visible. More precisely, the unsynchronized conflicting accesses form a C data race, so the program has undefined behavior; no particular output is guaranteed. (b) **ThreadSanitizer (TSan)** is appropriate; when the program is compiled and linked with `-fsanitize=thread` and run, it reports the data race.)*

Rubric: 2 for identifying a data race/undefined behavior; 2 for a coherent both-load-0 then store-1 interleaving; 1 for recognizing that one update is lost and the illustrated result is 1; 1 for ThreadSanitizer (also accept Valgrind's Helgrind or DRD, which detect races at run time).

Instructor Coverage and Scoring Notes

- Every question contributes to exactly one section; do not split a question's marks across learning outcomes.
- The section subtotals are A–H = 8, 8, 20, 16, 14, 16, 12, and 6 points, respectively.
- Suggested time allocation: A = 4 min, B = 4 min, C = 12 min, D = 10 min, E = 8 min, F = 9 min, G = 8 min, H = 5 min (60 minutes total).
- Equivalent wording and equivalent valid commands should receive credit.
- For race-condition questions, a technically correct answer should note that a C data race causes undefined behavior; the requested interleaving illustrates a possible lost update, not a guaranteed output.

Per-Student Section Reporting Sheet

Section	Question(s)	Possible	Earned	Section %
A	1–2	8		
B	3–4	8		
C	5–9	20		
D	10–12	16		
E	13–14	14		
F	15–17	16		
G	18–19	12		
H	20	6		
Total	1–20	**100**		